

Control flow

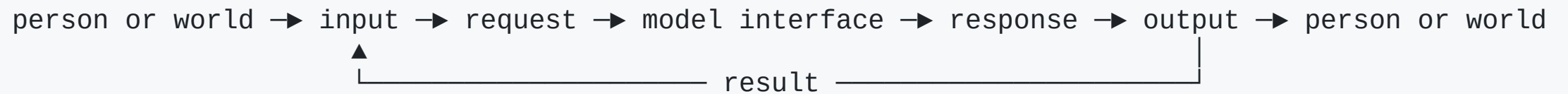
A hands-on course in building agents by building their harness

Lesson 3

Lesson 2 runs once, then stops

- Input, call, output, stop
- The model can ask for a tool, and the tool runs
- But the result never goes back to the model
- Something has to decide what happens next

Control flow is the path between the other four



- Input gathers, the model interface sends, output handles
- A tool's result is input again
- Control flow decides what happens at the end of the path

How you arrange them decides what you've built

- **Run once.** Input, call, output, stop. That's Lesson 2.
- **A chat loop.** Call, show the response, hand back to the person, repeat.
- **A workflow.** Your code decides the steps: draft, check, fix.
- **An agent loop.** While the model asks for tools, run them and call again.

The difference is who decides what happens next

- **Workflow:** your code decides
 - predictable and easy to test
 - only handles what you planned for
- **Agent:** the model decides
 - handles what you didn't plan for, and costs you predictability

Control flow sequences and terminates

- **Sequences:** passes what one primitive produced to the next
- **Terminates:** decides when to stop, or when to hand back to a person

A loop with no clear way to end is a bill with no clear way to end.

control_flow.py: a workflow, where your code decides

```
def workflow(input):  
    draft = ask(f"Do this task. Reply with only the result.\n\n{input}")  
    review = ask(f"Task:\n{input}\n\nDraft:\n{draft}\n\n...")  
    output = ask(f"Task:\n{input}\n\n...Rewrite the draft...")
```

(abridged: the prints and most of each prompt cut)

- Three calls, always in the same order: draft, review, rewrite
- The model writes every word, but never picks the next step

control_flow.py: an agent, where the model decides

```
while True:
    output = call(messages, tools=tools)
    messages.append({"role": "assistant", "content": output})
    ...
    done = subprocess.run(block.input["cmd"], shell=True, ...)
    input.append({... , "content": done.stdout or f"(exit {done.returncode})"})
    if not input: break
    messages.append({"role": "user", "content": input})
```

(abridged)

- The model picks each next step by asking for a tool
- `if not input: break`: no tool request means the model is done

Run the workflow: it rewrites even when it needn't

```
uv run lessons/03-control-flow/control_flow.py workflow "write one sentence about agent harnesses in which every word has exactly five letters, at least eight words long"
```

```
--- draft ---  
Agent tools guide model steps while small loops check every state.  
  
--- review ---  
The draft meets every requirement, so nothing needs fixing.  
...  
--- final ---  
Agent tools guide model steps while small loops check every state.
```

(shortened)

Run the agent: it picks its own steps

```
uv run lessons/03-control-flow/control_flow.py agent "which folder in lessons has the most files? answer in one English sentence"
```

```
$ cd lessons 2>/dev/null && for d in */; do echo "$(find "$d" -type f | wc -l) $d"; done | sort -rn | head -5
10 02-input-and-output/
9 03-control-flow/
7 04-context/
7 01-model-interface/
```

The `02-input-and-output` folder in `lessons` has the most files, with 10.

quark.py: the agent loop, plus a chat mode

```
chat = len(sys.argv) < 2

# — control flow —————
messages = [{"role": "user", "content": input}]

while True:
    output = call(show, max_tokens=16384, tools=tools, messages=messages).content
    messages.append({"role": "assistant", "content": output})
```

- `chat` is true when nothing came in on the command line
- Each pass of `while True:` is one call to the model
- `messages` carries everything so far: the simplest context, working memory

Tool results become the next input

```
input = []  
for block in output:                                # output: run tool requests  
    if block.type == "tool_use":  
        print(f"$ {block.input['cmd']}")  
        done = subprocess.run(block.input["cmd"], shell=True, ...)  
        print(done.stdout)  
        input.append({... , "content": done.stdout or f"(exit {done.returncode})"})
```

(abridged: two long lines cut to `...`; the last one is commented "input: from the world")

No tool request means it's done

```
if input:
    messages.append({"role": "user", "content": input})
    continue
if not chat or (input := read("\n> ")) == "/q":
    break
messages.append({"role": "user", "content": input})
```

- `if input: ... continue` is the agent loop: results go in, call again
- No input from the world is termination: no step count, no magic word
- `chat` gives one loop two modes: run to completion, or hand back with `>`

Run it: two calls, one answer

```
uv run lessons/03-control-flow/quark.py "which quark.py in the lessons
folder is the longest? answer in one sentence"
```

```
$ find . -path "*lessons*" -name "quark.py" -exec wc -l {} + | sort -n | tail -5
  12 ./lessons/01-model-interface/quark.py
  38 ./lessons/02-input-and-output/quark.py
  51 ./lessons/03-control-flow/quark.py
 239 ./lessons/04-context/quark.py
 340 total
```

The longest one is `./lessons/04-context/quark.py`, at 239 lines.

Other things we could do

- **Its shape:** single call, chat, workflow, agent loop, agents handing off, a loop woken by a message or a schedule
- **When it stops:** model stops asking, step count, spending limit, refusal, a person says stop
- **Who it hands back to:** a person, another program, or no one
- **What runs at once:** one model call, or several

[LangGraph](#) is a product built around this primitive alone.

Two ways to stop that quark leaves out

- **A step limit:** stop after, say, ten calls, even if the model is still asking
- **A refusal:** if `stop_reason` is `"refusal"`, the model declined; going around again won't help
- Either way, say which way it stopped

"Done", "out of steps" and "declined" look the same from outside if it doesn't.

Five workflows come up most

- **Prompt chaining:** a fixed sequence of calls
- **Routing:** one call sorts the input, your code picks the path
- **Parallelization:** calls at the same time, then combined
- **Orchestrator–workers:** one call splits the task, others do the pieces
- **Evaluator–optimizer:** one call produces, another judges, repeat

From Anthropic's [Building effective agents](#).

The rule: sequence the four, and decide when to stop

Arranged differently, the same four pieces are a single call, a chatbot, an agent, or a workflow.

What control flow never does

- Get the request to the model and the response back: **model interface**
- Gather what goes in: **input**
- Handle the response: **output**
- Decide what each request holds, even `messages`: **context**

Control flow only decides what runs, in what order, and when to stop.

What's missing: it's an agent that knows nothing

- Not where it is, what day it is, or who it is
- Not what you told it yesterday
- And `messages` grows every pass, until the model can't read it all

Something has to decide what the model sees.

Next: Context

Lesson 4